

An Open-Source Platform for High-Performance Non-Coherent On-Chip Communication

Andreas Kurth^{id}, *Student Member, IEEE*, Wolfgang Rönninger^{id}, Thomas Benz^{id},
Matheus Cavalcante, *Student Member, IEEE*, Fabian Schuiki^{id},
Florian Zaruba^{id}, *Student Member, IEEE*, and Luca Benini^{id}, *Fellow, IEEE*

Abstract—On-chip communication infrastructure is a central component of modern systems-on-chip (SoCs), and it continues to gain importance as the number of cores, the heterogeneity of components, and the on-chip and off-chip bandwidth continue to grow. Decades of research on on-chip networks enabled cache-coherent shared-memory multiprocessors. However, communication fabrics that meet the needs of heterogeneous many-cores and accelerator-rich SoCs, which are not, or only partially, coherent, are a much less mature research area. In this work, we present a modular, topology-agnostic, high-performance on-chip communication platform. The platform includes components to build and link subnetworks with customizable bandwidth and concurrency properties and adheres to a state-of-the-art, industry-standard protocol. We discuss microarchitectural trade-offs and timing/area characteristics of our modules and show that they can be composed to build high-bandwidth (e.g., 2.5 GHz and 1024 bit data width) end-to-end on-chip communication fabrics (not only network switches but also DMA engines and memory controllers) with high degrees of concurrency. We design and implement a state-of-the-art ML training accelerator, where our communication fabric scales to 1024 cores on a die, providing 32 TB/s cross-sectional bandwidth at only 24 ns round-trip latency between any two cores.

Index Terms—On-chip interconnection networks, multi-core multiprocessors, multiple data stream architectures, VLSI

1 INTRODUCTION

ON-CHIP networks are the primary means of communication inside modern multi- and many-core processing SoCs [1], [2], [3], [4]. As the number of cores, the heterogeneity of components, and the on- and off-chip bandwidth continue to grow to meet ever higher application demands, on-chip networks continue to gain importance. Decades of research on on-chip networks were instrumental for breakthroughs in scalability of homogeneous shared-memory multiprocessors, and a continuation of this research is necessary to realize the full potential of many-core accelerators and accelerator-rich heterogeneous SoCs.

Ideally, SoC designers could compose on-chip networks from a platform of components according to the requirements of their application. The central design goals of such a platform are: **(G1)** Elementary, modular components that can implement any topology and that separate concerns such as routing and buffering. **(G2)** Parametrizable components (e.g., data width, transaction concurrency) to cover a large design space. **(G3)** Bridging components to connect heterogeneous SoC elements (e.g., GPU SMs, DMA engines, and domain-

specific accelerators) and their subnetworks, each with unique, application-driven latency and bandwidth requirements. **(G4)** Compliance with an industry-standard protocol for extensibility, third-party compatibility, and verifiability. **(G5)** Detailed characterization of the complexity and trade-offs of the components in terms of performance versus cost (area, power) to guide design and optimization efforts.

Commercial offerings that meet (parts of) these goals exist from multiple vendors (details in Section 5), but their microarchitecture, complexity, and performance are well-guarded trade secrets. Research has also worked toward those goals (details in Section 5), but, to the best of our knowledge, an end-to-end platform for non-coherent on-chip communication that meets the needs of heterogeneous SoCs has not been presented yet in open literature and is not available as open-source hardware.

In this work, we fill this gap with these contributions:

- 1) We present a modular, topology-agnostic **(G1)**, high-performance on-chip communication platform of parametrizable components **(G2)** for a state-of-the-art, industry-standard protocol **(G4)** (Section 2). The components include bridges and converters to link subnetworks with different bandwidth and concurrency properties **(G3)**. We publish the modules of our platform, implemented in industry-standard SystemVerilog, under a permissive open-source license for research and industrial usage.
- 2) We discuss microarchitectural trade-offs and timing/area characteristics of the modules in our platform **(G5)**, both theoretically/asymptotically and with topographical synthesis results (Section 3). We show that our modules can be composed to build

• Andreas Kurth, Wolfgang Rönninger, Thomas Benz, Matheus Cavalcante, Fabian Schuiki, and Florian Zaruba are with the Integrated Systems Laboratory, ETH Zürich, 8092 Zurich, Switzerland. E-mail: {akurth, wroenmin, tbenz, matheus.cavalcante, fschuiki, zarubaf}@iis.ee.ethz.ch.

• Luca Benini is with the DEIS, Università di Bologna, 40126 Bologna, Italy. E-mail: luca.benini@unibo.it.

Manuscript received 19 Apr. 2021; revised 7 July 2021; accepted 25 July 2021.
Date of publication 25 Aug. 2021; date of current version 11 July 2022.

(Corresponding author: Andreas Kurth.)

Recommended for acceptance by C. Silvano.

Digital Object Identifier no. 10.1109/TC.2021.3107726

high-bandwidth (e.g., 2.5 GHz and 1,024 bit data width), end-to-end on-chip communication fabrics (e.g., DMA engine to memory controller), with high degrees of concurrency (e.g., up to 256 independent concurrent transactions) and flexibility (e.g., 64-bit subnetworks).

- 3) We design and implement (post-P&R) a state-of-the-art many-core MLT accelerator in a modern 22 nm technology (Section 4), where our communication fabric scales to 1,024 cores on a die, which deliver more than 2 Tdpflop/s, providing 32 TB/s cross-sectional bandwidth at only 24 ns round-trip latency between any two cores.

We focus on non-coherent on-chip communication for two main reasons: First, coherent on-chip communication in homogeneous many-core processors has been studied extensively (see Section 5 for an overview). Second, many complex heterogeneous SoCs (e.g., mobile application SoCs [5], high-speed networking SoCs [6]) and massively parallel data processing architectures (e.g., GPGPUs [7]) are not or only partially cache-coherent.

This paper is organized as follows: We present the architecture of our on-chip communication platform in Section 2 and characterize its performance and complexity in Section 3. We then use our platform to design, implement, and evaluate the communication fabric of a state-of-the-art many-core MLT accelerator in Section 4. Finally, we compare with related work in Section 5 and conclude in Section 6.

2 ARCHITECTURE

Current on-chip communication is centered around the premise of high-bandwidth point-to-point data transfers. To fulfill this premise despite increasing point-to-point latency, three central traits of current on-chip communication protocols are: *burst-based transactions*, *multiple outstanding transactions*, and *transaction reordering*. Our design targets these central traits in general, so the concepts we present potentially apply to a wide range of modern on-chip protocols. More tangibly, we adhere to the latest revision (5) of the AMBA Advanced eXtensible Interface (AXI) [8]. AXI is one of the industry-dominant protocols and the only protocol with an open, royalty-free specification and a widespread adoption in current systems designed by many different companies. Other protocols with similar properties are discussed in Section 5.

Terminology and Protocol Essentials. A *module* is a distinct functional unit that has at least one on-chip network port. A *port* is a collection of input and output signals of a module. A port can be either a *master port*, on which the module initiates transactions, or a *slave port*, on which the module responds to transactions. One module can have multiple slave and master ports. We collectively call the five independently-handshaked channels connecting a master port to a slave port a *bundle*. Each *channel* consists of multiple is-directional payload signals and two signals for bi-directional flow control. A *beat* is the data transferred on one channel upon one handshake; it is the smallest unit of communication. We focus on *valid-ready* flow control, where the channel master drives the *valid* signal and the payload signals and the channel slave drives the *ready* signal (but other flow control schemes, e.g., credit-based, are possible). A

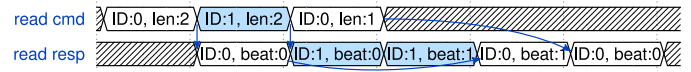


Fig. 1. Transaction ID and ordering example. Three read commands are issued; the first and last have the same ID, the one between has a different ID. This situation might occur, e.g., if the transactions are issued by different masters. The first response beat is for the first command. After that follow both response beats for the second command. This interleaving of response beats is allowed because the first two commands have different IDs, so their responses may be interleaved. Then follows the second response beat for the first command. The response beat for the last command comes at the end, because it must not come before the last response beat of the first command (which has the same ID).

handshake occurs when *valid* and *ready* are high on a rising clock edge. There are two essential rules in *valid-ready* flow control: **(F1) Stability Rule:** Once *valid* is high, *valid* and the payload must not change until the handshake occurs. **(F2) Acyclicity Rule:** The channel slave may depend on *valid* to be high before setting *ready* high, but the channel master may not depend on *ready* to be high before setting *valid* high.

Each transaction has a *direction* (read or write): A *write transaction* starts with one beat on the *write command channel* followed by one or multiple beats on the *write data channel* and ends with a single beat on the *write response channel*. A *read transaction* starts with one beat on the *read command channel* and ends with one or the last of multiple beats on the *read response channel*. A transaction is *outstanding* in the time interval starting with the handshake of the command beat and ending with the handshake of the (last) response beat. Each transaction has a numeric *ID*. IDs define the order of transactions and beats according to the following rules: **(O1) Inter-Transaction Ordering:** Any two transactions in the same direction and ID are ordered. **(O2) Response Ordering:** Any two responses with the same direction and ID must be in the same order as their commands. **(O3) Write Beat Ordering:** Write data beats do not have an ID and are therefore always ordered. An example of IDs and their ordering is shown in Fig. 1. The rest of this section discusses the microarchitecture and design trade-offs of our on-chip communication platform, from elementary components through all essential interconnecting modules to endpoints of increasing complexity.

2.1 Elementary Components: Network (De)Multiplexers

Our network multiplexers and demultiplexers are the elementary components that join multiple ports to one and split one port into multiple, respectively. In doing so, they must adhere to the relations between the channels and to the ordering rules **(O1–3)**. They are obviously used to build network junctions (e.g., crossbars), but they can be reused far beyond that because they implement a central part of the communication protocol. In fact, these elementary components are essential for almost all modules of our platform and can be used to design custom endpoints without having to deal with all protocol intricacies. Each of our *network (de) multiplexers* contains simple *logic (de) multiplexers*, but it also contains other components to implement the protocol. In the remainder of this article, ‘(de)multiplexer’ refers to a *network (de) multiplexer* unless explicitly preceded by ‘logic’.

2.1.1 Network Multiplexer

The multiplexer, which connects multiple slave ports to one master port, consists of multiplexing components for the

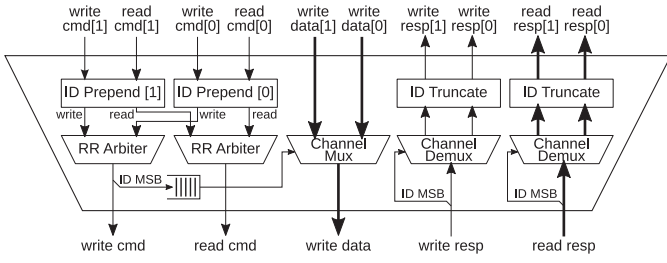


Fig. 2. Architecture of our network *multiplexer*, drawn with two slave ports (at the top). Here and in Fig. 3, ‘Channel (De)mux’ refer to logic (de)multiplexers with additional handshake logic for one channel. In all later figures, ‘Mux’ refers to the entire module in Fig. 2 and ‘Demux’ refers to the entire module in Fig. 3.

forward channels and demultiplexing components for the backward channels. The complexity lies in *demultiplexing* the backward channels, because the multiplexer needs the information to which output a beat on a backward channel must be routed. Multiplexing the command channels simply requires the selection of a *valid* beat, with the restriction that a selection must be stable once made (F1).

Our multiplexer architecture is shown in Fig. 2. We first prepend the ID of each command beat with the number of the slave port. We then select among beats on the command channels with round-robin (RR) arbitration trees. For writes, the decision is forwarded through a first-in first-out buffer (FIFO) to a multiplexer for the write data beats, which is sufficient due to (O3). As commands out of our multiplexer carry the input port information in the most significant bits (MSBs) of their ID, routing responses is as simple as demultiplexing based on the MSBs and then truncating the ID to the original width. Another key advantage is that transactions with the same ID from any two different slave ports remain independent, so (O1) does not restrict communication through our multiplexer. Note that *channel demultiplexing* means the payload is the same for all demux outputs and only the handshake signals are (de)multiplexed.

Alternative multiplexer architectures could do without extending the ID, for example by allowing only transactions with different IDs concurrently or by remapping IDs internally. However, the former restricts communication, and the latter significantly increases the complexity of the multiplexer. Nonetheless, some network modules grow exponentially in complexity with the ID width. We have a modular solution to this challenge with the ID width converters discussed in Section 2.3.

2.1.2 Network Demultiplexer

The demultiplexer, which connects one slave port to multiple master ports, is more complex than the multiplexer due to the ordering rules: When the demultiplexer gets two commands with the same ID and direction (O1) that go to two different master ports, it must deliver the corresponding responses in the same order (O2). After the demultiplexer, however, transactions on different master ports are independent, so the demultiplexer cannot rely on the order of downstream responses to fulfill (O2).

Our demultiplexer architecture, shown in Fig. 3, solves this by enforcing that all concurrent transactions with the same direction and ID target the same master port. For example, when a write with ID *A* targets master port 0, it is

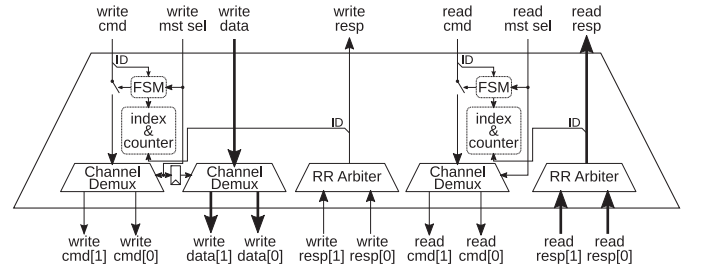


Fig. 3. Architecture of our network *demultiplexer*, drawn with two master ports (at the bottom).

only forwarded if no writes with ID *A* to master ports other than 0 have outstanding responses; otherwise, the write must wait. To track this information, the demultiplexer contains one counter and one index register per ID and direction. Commands that fulfill the aforementioned requirement increase the counter; the (last) response decreases the counter. A channel register between the write command channel and the demultiplexer of the write data channel stores the master port index of an ongoing write burst while the command channel is independently handshaked (F1). Write commands and data bursts are sent in lockstep due to (O3); without this restriction, the write command and data channels could deadlock downstream. The multiple read and write response channels are joined through a round-robin arbitration tree.

Alternative demultiplexer architectures could do without requiring all concurrent transactions with the same direction and ID to target the same master port, for example by remapping IDs internally. However, this significantly increases the complexity of the demultiplexer, which would have to reorder responses internally to fulfill (O2). Instead of introducing this complexity, we let a master use different IDs for different endpoints if it can handle out-of-order responses.

Compared with a 1-to-N crossbar, the demultiplexer has a fundamental advantage concerning how transactions are routed: With the crossbar, the *address* of a transaction determines to which master port it is routed. With the demultiplexer, the *select inputs* (one for reads, one for writes) determine to which master port a transaction is routed. This means a module instantiating the demultiplexer can freely decide which submodule handles a transaction. That decision does not even have to be based on the properties of the transaction but could, for example, be a function of the state of the module. This difference implies that the demultiplexer is a more universal elementary component than a 1-to-N crossbar.

Logic demultiplexers are so universally used in digital circuits that our *network* demultiplexer may seem like a trivial sequel. However, as the architecture depicted in Fig. 3 and described in this section shows, our demultiplexer handles crucial and complex parts of the protocol ((O1–3), (F1)). Thus, even though our demultiplexer simply takes a select signal to route transactions, it unburdens user modules from dealing with intricacies of the protocol while it enables them to arbitrarily route transactions to submodules or ports.

2.2 Network Junctions: Crossbars and Crosspoints

2.2.1 Crossbar

The elementary components in Section 2.1 can be combined to form a fully-connected crossbar, shown in Fig. 4, where

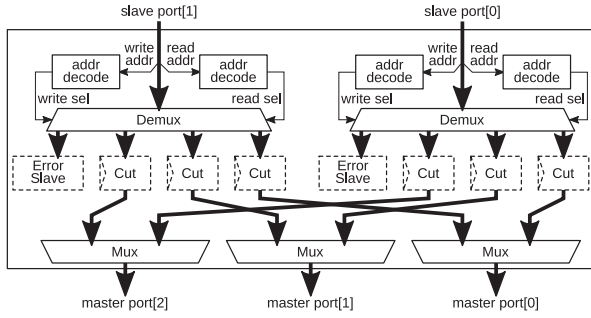


Fig. 4. Architecture of our *crossbar*, drawn with two slave and three master ports. Each fat arrow represents a bundle, with the arrow head pointing in the direction of the command channels. Components with dashed outline are optional.

each slave port has a dedicated connection to each master port.

At each slave port, two address decoders (one for reads, one for writes) drive the selection signals of a demultiplexer. In the standard configuration, all slave ports use the same addresses for one master port, but different configurations would be possible. There are two alternatives for handling transactions to an address that is not defined in a decoder. First, one master port can be defined as default port. This is useful, for example, in a hierarchical topology where each downlink has a specific range of addresses and any address outside the downlink addresses is sent to higher hierarchy levels through the uplink. Second, one can instantiate an error slave, which terminates all transactions with protocol-compliant error responses. These two alternatives can be selected per slave port with a synthesis parameter.

Optional pipeline registers can be inserted on all or some of the five channels of each internal bundle. These registers cut all combinational signals (including handshake signals), thereby adding a cycle of latency per channel and pipelining the crossbar so its critical path is no longer than that of the demultiplexer or multiplexer. These pipeline registers can be added without risking deadlocks, but this is not trivial: Of the four Coffman conditions [9], (1) Mutual Exclusion is fulfilled on the write data channel after the multiplexer, (2) Hold and Wait is fulfilled as each pipeline register must hold its value once filled, (3) No Preemption is fulfilled by (O3) on the write data channel, and (4) Circular Wait would be fulfilled by round-robin arbitration of write command and data beats. However, the demultiplexer breaks condition (4) by restricting write commands to be issued in lockstep with write data bursts (i.e., the next write command is only issued after the previous write data burst has completed), thereby preventing deadlocks despite pipeline registers, which introduce condition (2).

2.2.2 Crosspoint

As the multiplexers in the crossbar expand the ID width, the master ports of the crossbar have a wider ID than the slave ports. This prevents the direct use of our crossbar as nodes in a regular on-chip network where each node (also called “router” or “switch”) has isomorphous slave and master ports. To solve this problem, we introduce a crosspoint.

Our crosspoint, shown in Fig. 5, has three additional properties over the crossbar that make it better suited for composing arbitrary regular on-chip topologies. First, it

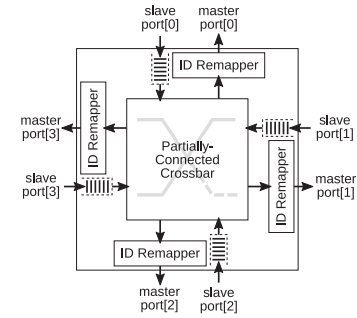


Fig. 5. Architecture of our *crosspoint*, drawn with four slave and master ports. Each arrow represents a bundle, with the arrow head pointing in the direction of the command channels. The input queues are optional.

contains a crossbar that is not necessarily fully connected: The connection between any slave and master port can be omitted with a synthesis parameter. This is useful to prevent routing loops when a module has both a master and a slave port into the crosspoint, and it minimizes the physical resources on links that would be unused. All flow and arbitration control logic of the crosspoint is inside the crossbar. Second, the crosspoint contains an ID remapper (Section 2.3.1) on each master port, which reduces the ID width to that of the slave ports. Thus, the slave and master ports of each crosspoint are isomorphous. Third, an input queue of configurable depth can be enabled for each slave port to reduce backpressure in mesh topologies.

2.3 Concurrent Transactions: ID Width Converters

The ID of transactions is central to their ordering (O1–2). Essentially, the commands and responses of any two transactions can be independently reordered if they have different IDs. This makes a high number of possible IDs attractive to prevent bottlenecks due to ordering constraints. However, tracking a high number of IDs is complex for network components (e.g., demultiplexer Sections 2.1.2 and 3.1.2).

ID width converters are the on-chip network designer’s instrument to balance the number of independent concurrent transactions versus circuit complexity. We focus on *reducing* the ID width (as extending it is trivial). There are two first-order parameters for ID reduction: the width of IDs at the output, O , and the maximum number of unique IDs at the input, U . The relation between O and U determines whether all transactions that were independent at the input remain independent at the output: If $U \leq 2^O$, every unique ID at the input can be represented by a unique ID at the output, therefore retaining transaction independence. This means the sparsely used input ID space can be ‘compressed’ to a narrower, densely used output ID space by *remapping* IDs (Section 2.3.1). If $U > 2^O$, there are not enough output IDs to represent all U unique IDs. This means some transactions with originally different IDs will have to be mapped to the same ID, thereby *serializing* them (Section 2.3.2).

2.3.1 ID Remapper

Our ID remapper, shown in Fig. 6, remaps IDs with one table per direction. The table has as many entries as there are unique input IDs, and it is indexed by the output ID. Each table entry has two fields: the input ID and a counter

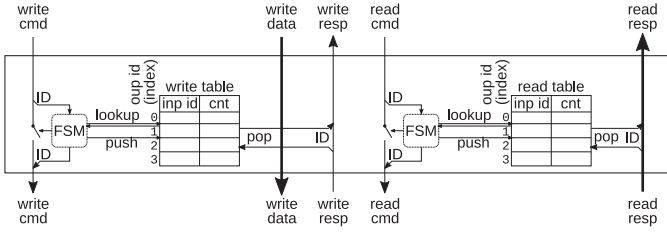


Fig. 6. Architecture of our *ID remapper*, drawn with up to four unique concurrent IDs (per direction).

that records how many transactions with the same ID are in flight. The counter is incremented on command handshakes and decremented on (last) response handshakes. The mapping from input to output IDs is injective. Obtaining the input ID from an output ID (to remap responses) is as simple as indexing the table. Determining the output ID for an input ID (to remap commands) requires a comparison of the input ID to all IDs in the table. If the table currently contains an entry for the input ID, the same output ID must be used (**O1**). If the table does not currently contain an entry for the input ID, the output ID is the index of the next free table entry.

Alternative ID remapper architectures could feature an additional table indexed by input IDs to look output IDs up. However, under the assumption of the remapper that the input ID space is sparse, such an additional table would be mostly empty. Therefore, it would be a poor usage of hardware resources and we omit it at the cost of a longer ID translation path, which could be pipelined.

2.3.2 ID Serializer

If the number of unique IDs at the input of the ID width converter, U , exceeds the number of available IDs at the output, 2^O , both the input and the output ID space are densely used. In this case, it is not possible to retain the uniqueness of all IDs during conversion, and we call the transformation that imposes additional ordering *serialization*. Serialized transactions still have concurrently outstanding commands, but they are now required to be handled in-order.

Our ID serializer, shown in Fig. 7, transforms IDs with one FIFO per direction and master port ID. At the slave port of the serializer, a demultiplexer assigns commands to one of the FIFO submodules through a combinational function f of the transaction ID (e.g., the ID modulo the number of master port IDs). The demultiplexer is a reduced configuration of our network demultiplexer (Section 2.1.2) without ID counters because f assigns identical IDs to the same master port (and thus the same output ID (**O1**)). In each FIFO submodule, the ID of a command is pushed into a FIFO and then truncated to zero. This FIFO reflects the transaction ID in responses (**O2**), and the last response of a transaction pops from the FIFO. After the FIFOs, an instance of our network multiplexer (Section 2.1.1) assigns each transaction the index of its FIFO and merges the commands to the single master port of the ID serializer.

Alternative ID serializer architectures could use one memory where one linked list per master port ID is stored for ID reflection. This would allow to dynamically grow queues in memory rather than statically provisioning hardware resources to accommodate a fixed maximum of transactions per

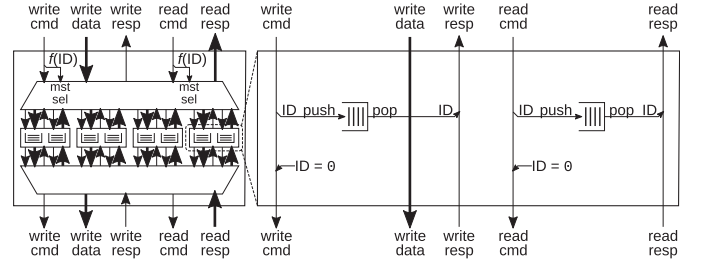


Fig. 7. Architecture of our *ID serializer*, drawn with four master port IDs (per direction).

master port ID. However, pushing and popping IDs from this memory is on the critical path of the serializer, so we prefer the architecture with multiple FIFOs.

2.4 Data Width Converters

The data width of network components depends on their bandwidth requirements. For instance, the master port of a high-performance DMA engine might have 512 bit data width while that of a 64-bit processor core typically has 64 bit. This extends to subnetworks, e.g., separate networks for the DMA engine and the cores. However, as subnetworks with different data widths are joined, e.g., at endpoints such as memories, data width converters (DWCs) are required to convert between data widths. DWCs can be either *upsizers*, converting from narrow to wide, or *downsizers*, converting from wide to narrow. Although similar in purpose, up- and downsizer are not fully symmetric. In fact, the upsizer has higher performance requirements than the downsizer, since it must utilize the higher-bandwidth network as much as possible to minimize the impact on other components on the high-bandwidth network.

2.4.1 Data Upsizer

A data upsizer has a narrow slave port of data width D_N and a wider master port of data width D_W . In the simplest operating mode, pass-through, the upsizer does lane selection on read responses (Fig. 8a), selecting a slice of a wide incoming word, and lane steering on write data, aligning narrow incoming data into the wider outgoing word (Fig. 8b). In pass-through mode, the upsizer does not change the number of bytes transferred in each beat. This can be required by transaction attributes (e.g., to device memory). In terms of performance, however, this underutilizes the high-bandwidth network, which inherits the throughput of its low-bandwidth counterpart. Utilization can be increased by reshaping incoming bursts with many narrow beats into bursts with fewer wide beats: several narrow write data beats are packed into one wide beat, and one wide read response beat is serialized into several narrow beats.

Our data upsizer, shown in Fig. 8c, is capable of upsizing between interfaces of any data width. It is composed by two modules, read and write upsizers, that perform lane selection and steering, besides deciding whether to upsize the transaction based on its properties. Due to (**O3**), only one write upsizer is needed, containing a buffer of width D_W to perform data packing. On the read response channel, the data upsizer handles a certain number of outstanding read transactions in parallel. Each incoming read transaction is

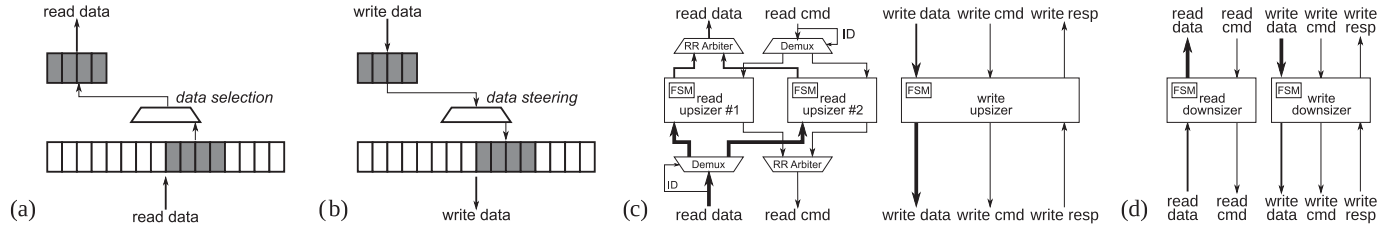


Fig. 8. Architecture of our *data width converters (DWCs)*. (a) Data selection in the read response and (b) data steering in the write data channel of the upsizer. (c) Upsizer, drawn with two outstanding read transactions. (d) Downsize.

assigned an idle read upsizer, unless there is an active upsizer handling a transaction with the same ID. For that case, we ensure (O1) by enforcing that incoming transactions with the same ID are handled by the same read upsizer. Each read upsizer has a D_W buffer to hold incoming beats. This avoids blocking the wide read response channel during serialization.

2.4.2 Data Downsize

A data downsize has a wide slave port of data width D_W and a narrower master port of data width D_N . In the simplest operating mode, pass-through, the downsize does steering on the read data channel and selection on the write data channel, symmetrical to the base operations of the data upsizer. Our downsize, shown in Fig. 8d, differs from the upsizer in two key points: First, the downsize has lower performance requirements than the data upsizer, since it connects to a lower-bandwidth subnetwork, e.g., peripherals. This means it does not need to support multiple outstanding reads. Second, when downsizing, the downsize converts few wide beats into multiple narrow beats. It is possible that the resulting burst is longer than the longest buffer allowed by the protocol. In this case, the downsize needs to break the incoming burst into a sequence of bursts. To handle this corner case, among others, the control logic of the read and write downsizers is more complex than those in the upsizer.

2.5 Clock Domain Crossing

An on-chip network can span multiple clock domains, yet all our modules have a single clock input¹ – except one: the clock domain crossing (CDC) has two clock inputs, one to which all signals of its slave port are synchronous and one for its master port. The CDC can be placed between any two modules in different clock domains. This enables the creation of independently-clocked subnetworks as well as the connection to endpoints that provide their own clock. In our CDC, shown in Fig. 9, each channel goes through a CDC FIFO, which has two Gray-coded counters: one for pushing the FIFO in one clock domain and one for popping from the FIFO in the other clock domain. The implementation follows well-established CDC principles [10], [11], [12].

2.6 Data Movement: DMA Engine

Transferring large amounts of data at high bandwidth requires dedicated components for data movement called *direct memory access (DMA) engines*. Our DMA architecture

is designed to be modular, dividing the unit into two parts: a system-specific frontend and a backend implementing the data movement within the on-chip interconnect. We define a simple, yet well-defined interface uniting both parts: a one-dimensional and contiguous memory block of arbitrary length, source, and destination address, called *1D transfer*. We chose this interface abstraction because 1D transfers map very well to burst-based transactions. More complex transfers, such as multi-dimensional or strided accesses, are decomposed by the frontend into 1D transfers. As the frontend is highly system-specific, we will not discuss it.

In the backend, the *burst reshaper*, shown in Fig. 10a, divides the arbitrary-length 1D transfers into protocol-compliant bursts (adhering to, e.g., address boundaries and maximum number of beats). On arrival of a new 1D transfer, the burst converter loads length, source address, and destination address into internal registers. The *burst boundaries* process determines the number of bytes that can be requested in the next burst. With this, the burst reshaper calculates the address of the next burst and the remaining bytes left in the 1D transfer. Each protocol-compliant burst is then translated by the *data mover* unit, shown in Fig. 10b, into a read and a write command as well as a read and a write data job. The commands are issued as beats on the command channels. The data jobs are forwarded to the data path. The *data path*, shown in Fig. 10c, receives read data beats, realigns the data to compensate for different byte offsets between the read and write data streams, and issues write data beats. The data path consists of two independent processes. The read process is realigns and buffers incoming data. If a burst starts on an unaligned address, some leading bytes (“head”) in the first beat are invalid and are masked. Similarly, a burst may end on an unaligned address, in which case some trailing bytes in the last beat (“tail”) need to be masked. The write process drains data from the buffer as soon as it is available and masks it according to the destination address offset with the strobe signal of the write data channel.

2.7 On-Chip Memory Controllers

On-chip memories are an important class of endpoints for on-chip network transactions. In this section, we describe

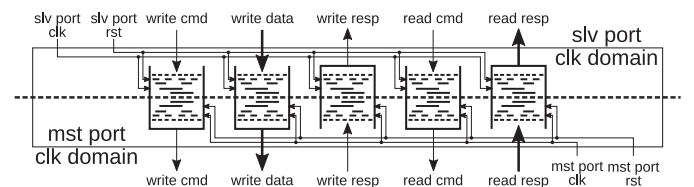


Fig. 9. Architecture of our *clock domain crossing (CDC)*. Each channel goes through a CDC FIFO, which has two Gray-coded counters.

1. For modules with a single clock input, we do not draw the clock input and clock wires to all sequential cells in the block diagram in order to not overcrowd the diagram.

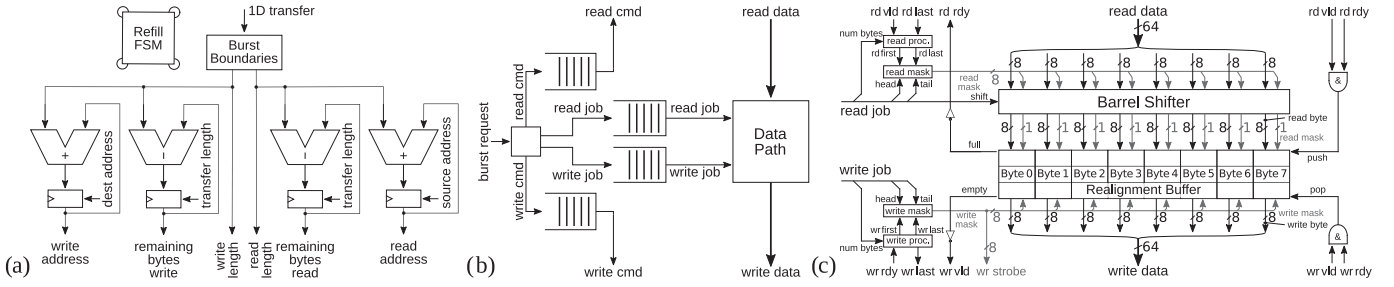


Fig. 10. Architecture of our *DMA engine*. (a) Burst reshapers. (b) Data mover. (c) Data path, drawn for 64 bit data width.

two memory controllers through which standard single-port static random access memory (SRAM) macros can be connected to the on-chip network.

2.7.1 Simplex Memory Controller

The architecture of our simplex on-chip memory controller is shown in Fig. 11. *Simplex* in this context means that the controller in each clock cycle can either read or write memory, as is natural for a single-port SRAM. The memory controller first translates read commands and write commands plus write data into memory commands. An arbiter then forwards either a read or a write memory command per clock cycle. This arbiter optionally takes quality of service (QoS) attributes of a command into account and can prioritize write beats, which cannot be interleaved due to (O3), over read beats. A stream fork unit splits address and data, which go to the memory interface, and meta data (e.g., the transaction ID), which are used by the memory controller to form responses in the network protocol. A converter translates the address and data stream into memory interface signals (with stream flow control on the command and no handshaking on the response path). The memory responses are then joined with meta data to form read or write responses, which are finally issued on the corresponding network response channel.

The simplex memory controller cannot achieve the full bidirectional bandwidth of the duplex on-chip network interface, which has separate channels for read and write data. The duplex memory controller removes this limitation.

2.7.2 Duplex Memory Controller

The architecture of our duplex memory controller is shown in Fig. 12. To saturate the read and write data channels of the on-chip network simultaneously (thus *duplex*), this memory controller has at least two independent memory master ports as well as one simplex controller for writes and one for reads.

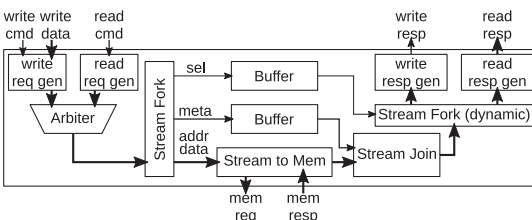


Fig. 11. Architecture of our *simplex on-chip memory controller*, with the on-chip network slave port at the top and the memory master port at the bottom. The memory master port has the same data width as the network slave port.

A network demultiplexer statically routes all writes through the left controller and all reads through the right controller. The unused resources inside both simplex controllers are optimized away during synthesis. A logarithmic memory interconnect then routes each command to one of the memory master ports, which are address-interleaved.

The duplex memory controller can fully saturate both the read and the write data channel of the on-chip network in the absence of conflicts on the memory ports. However, irregular traffic (e.g., misaligned addresses, mixed wide and narrow beats) can give rise to a significant conflict rate. To reduce conflicts, the *banking factor* (i.e., the number of memory master ports per network slave port) can be increased to any integer higher than 2 (at the cost of more wide and shallow SRAM macros when the memory capacity is to remain constant).

3 IMPLEMENTATION RESULTS

This section provides quantitative and asymptotic complexity results for our network modules. These results are essential for architects to assess the feasibility and strike trade-offs in the design of on-chip networks. Our findings are summarized in Section 3.8. Until there, this section discusses implementation results to derive the findings.

We implement the modules presented in Section 2 in GlobalFoundries' 22nm fully-depleted silicon-on-insulator (GF22FDX) technology, using a ten-metal stack and eight track SLVT/LVT flip-well standard cells characterized at typical conditions (0.8 V, 25°C). We synthesize with Synopsys DesignCompiler 2019.12 using topographical mode, so physical place-and-route constraints, dimensions, and delays are taken into account. For the isolated implementation of the modules, each input is driven by a D-flip-flop (FF), and each output drives a D-FF. Unless we vary it in the evaluation, we set the address and data width to 64 bit and the slave port ID width to 6 bit. Before undergoing synthesis,

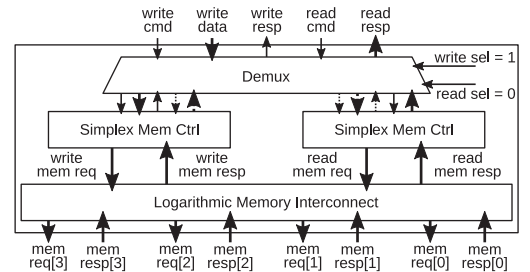


Fig. 12. Architecture of our *duplex on-chip memory controller* with four address-interleaved memory master ports.

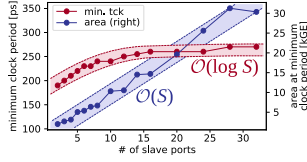


Fig. 13. Minimum clock period and corresponding area of our network multiplexer in GF22FDX for 2 to 32 slave ports and 6 ID bits.

all modules have been verified for protocol compliance in RTL simulation under extensive directed and constrained random verification tests.

3.1 Elementary Components: Network (De)Multiplexers

3.1.1 Network Multiplexer

The critical path of the multiplexer goes through from a slave port command channel through the arbitration tree on its handshake signals and the multiplexers on its payload signals to a master port command channel. For S slave ports, it scales with $\mathcal{O}(\log S)$ due to the logarithmic depth of the arbitration tree and the multiplexers. The area scales $\mathcal{O}(S)$ due to the linear area of the arbitration tree and the multiplexers. The area is further linear in the ID width and the maximum number of write transactions due to the FIFO between write command and data channel, but this part is usually negligible. Fig. 13 shows the area and timing characteristics of our multiplexer: for 2 to 32 slave ports, the critical path increases logarithmically from 190 to 270 ps, and the area increases linearly from 2 to 30 kGE.

3.1.2 Network Demultiplexer

The critical path of the demultiplexer goes from a command channel at the slave port through ID lookup to a command channel on one of the master ports. It scales with $\mathcal{O}(M)$ as the channel demultiplexers grow linearly in area with the master ports and topographical synthesis takes the distance increase into account. The area scales with $\mathcal{O}(M)$ due to the linear area of the arbitration trees and the channel demultiplexers. The ID width I is critical for the demultiplexer: the area scales with $\mathcal{O}(2^I)$ due to the exponential number of counters (one for every possible ID), and the critical path scales with $\mathcal{O}(I)$ because every ID bit adds a multiplexer level in the indexing logic of the counters. Fig. 14 shows the area and timing characteristics of our demultiplexer: For 2 to 32 master ports and 6 ID bits (Fig. 14a), the critical path increases linearly from 330 to 430 ps, and the area increases linearly from 22 to 38 kGE. The curve is non-monotonic mainly in two points, where the synthesizer selects disproportionately strong and large buffers to reach the target

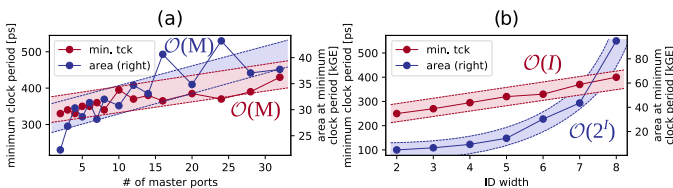


Fig. 14. Minimum clock period and corresponding area of our network demultiplexer in GF22FDX: (a) with 2 to 32 master ports and 6 ID bits, and (b) with 4 master ports and 2 to 8 ID bits.

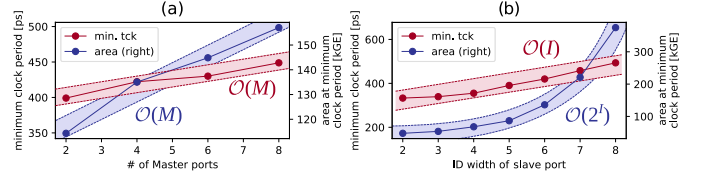


Fig. 15. Minimum clock period and corresponding area of our crossbar with 4 slave ports, fully connected and unpipelined, in GF22FDX: (a) with 2 to 8 master ports, 4 slave ports and 6 ID bits, and (b) with 4 master ports and 2 to 8 ID bits at the slave port.

frequency. For 4 master ports and 2 to 8 ID bits (Fig. 14b), the critical path increases linearly from 250 to 400 ps, and the area increases exponentially from 5 to 95 kGE. Depending on the ID width, the critical path can be significantly longer than in the multiplexer, so the demultiplexer will be the critical stage in a pipelined network junction.

3.2 Network Junctions: Crossbars and Crosspoints

3.2.1 Crossbar

For a fully-connected crossbar with S slave ports, M master ports and I bits at the slave port, the critical path is dominated by the demultiplexer, thus scales with $\mathcal{O}(M + I)$. The area is the sum of the area of the S demultiplexers and M multiplexers plus a small overhead for each slave port for address decoding and the error slave (when instantiated). The area thus scales with $\mathcal{O}(MS + 2^I S)$. Fig. 15 shows the area and timing characteristics of a fully-connected, unpipelined instance of our crossbar: For 4 slave ports, 2 to 8 master ports and 6 ID bits (Fig. 15a), the critical path increases linearly from 400 to 450 ps, and the area increases linearly from 111 to 156 kGE. As was the case for the demultiplexer (Section 3.1.2), the ID width of the slave ports has significant impact on the critical path and area of the crossbar. For 4 master and 4 slave ports and 2 to 8 ID bits (Fig. 15b), the critical path increases linearly from 340 to 460 ps, and the area increases exponentially from 42 to 390 kGE.

3.2.2 Crosspoint

The critical path of a fully pipelined crosspoint goes from the internal pipeline register of a master port into the table of an ID remapper. For M master ports (Fig. 16a), it scales with $\mathcal{O}(M)$ from 610 to 630 ns as topographical synthesis takes the area increase into account. The area also scales with $\mathcal{O}(M)$ but much more significantly from 243 to 587 kGE as the crossbar and the number of ID remappers scale linearly. Regarding the ID width I , the crosspoint is dominated by the demultiplexer: For 2 to 8 ID bits in a 4×4 configuration (Fig. 16b), the area scales with $\mathcal{O}(2^I)$ from 127 to 1181 kGE and the critical path scales with $\mathcal{O}(I)$ from 290 to 800 ps.

3.3 Concurrent Transactions: ID Width Converters

3.3.1 ID Remapper

The critical path of our ID remapper goes from the input ID through the ID equality comparators in the table, through a leading-zero counter (LZC) to determine the matching or the first free output ID, into a table counter entry. For an input ID width of I , up to U concurrent unique IDs (per direction), and up to T transactions per ID, it scales with

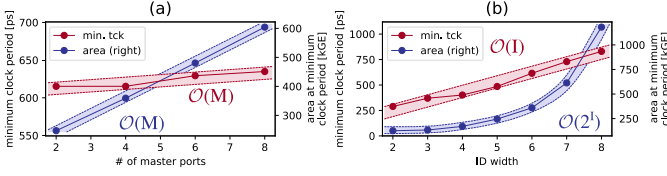


Fig. 16. Minimum clock period and corresponding area of our *crosspoint* with 4 slave ports, fully connected and pipelined, in GF22FDX: (a) with 2 to 8 master ports, 4 slave ports and 6 ID bits, and (b) with 4 master ports and 2 to 8 ID bits at the ports.

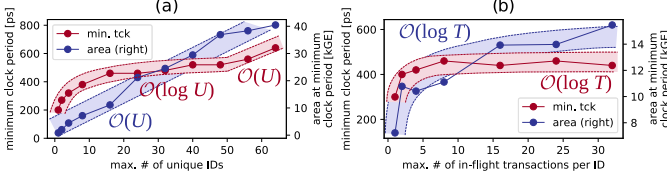


Fig. 17. Minimum clock period and corresponding area of our *ID remapper* in GF22FDX: (a) for 1 to 64 concurrent unique IDs and 8 transactions per ID, and (b) for 16 concurrent unique IDs and 1 to 32 transactions per ID.

$\mathcal{O}(\log I + \log U + \log T)$. The area is dominated by the tables, which have U entries with $I + \log_2 T$ bit each. Additionally, the LZCs have an area of $\mathcal{O}(U \log U)$. The total area thus scales with $\mathcal{O}(U(I + \log T + \log U))$. Fig. 17 shows the area and timing characteristics of our *ID remapper*: For $U = 1$ to 64 concurrent unique IDs and $T = 8$ transactions per ID (Fig. 17a), the critical path increases logarithmically from 200 to 520 ps until $U = 48$ and then linearly to 640 ps for $U = 64$ as path delays due to the linearly growing table start to dominate. The area increases linearly from 1 to 41 kGE. The highest (rightmost) configuration can remap up to 512 transactions in both directions with up to 64 unique IDs concurrently, but the area and critical path costs are quite high. In comparison, for $U = 16$ concurrent unique IDs and $T = 1$ to 32 transactions per ID (Fig. 17b), the critical path increases logarithmically from 300 to 440 ps, and the area increases logarithmically from 7 to 16 kGE. Thus, the highest (rightmost) configuration can also remap up to 512 transactions but with only up to 16 unique IDs concurrently, at a $2.6\times$ lower area and $1.5\times$ shorter critical path.

3.3.2 ID Serializer

The critical path of the ID serializer goes through the demultiplexer, the push side of the ID FIFO, and the arbitration tree in the multiplexer. For U_M IDs at the master port and T transactions per master port ID, it scales with $\mathcal{O}(\log U_M + \log T)$. The area scales with $\mathcal{O}(U_M + T)$ due to the linear area of all components in either U_M or T . Fig. 18 shows the area and timing characteristics of our *serializer*: For $U_M = 1$ to 32 IDs at the master port and $T = 8$ transactions per master port ID (Fig. 18a), the critical path increases logarithmically from 195 to 410 ps, and the area increases linearly from 2 to 190 kGE. Clearly, compressing a densely used ID space is expensive in terms of area. This cost can be reduced by fixing U_M at a low value and varying T : For $U_M = 4$ IDs and $T = 1$ to 32 transactions per ID at the master port (Fig. 18b), the critical path increases logarithmic from 245 to 280 ps, and the area increases linearly from 15 to 51 kGE. 128 concurrent transactions (in both directions) could therefore be serialized with $U_M = 4, T = 32$ at $1.28\times$ less area and $1.29\times$ shorter critical path.

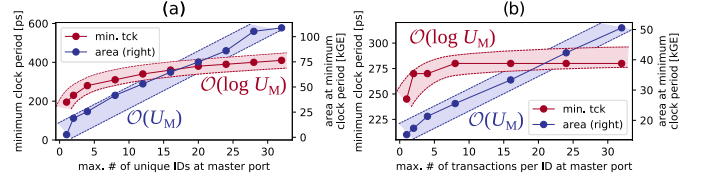


Fig. 18. Minimum clock period and corresponding area of our *ID serializer* in GF22FDX: (a) for 1 to 32 IDs at the master port and 8 transactions per master port ID, and (b) for 4 IDs and 1 to 32 transactions per ID at the master port.

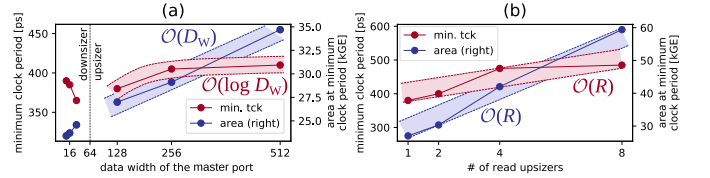


Fig. 19. Minimum clock period and corresponding area of: (a) our *data downsizer* and *upsizer*, considering a slave port data width of 64 bit and a master port data width from 8 to 512 bit, and (b) our *data upsizer*, considering a slave port data width of 64 bit, a master port data width of 128 bit, and 1 to 8 read upsizers.

3.4 Data Width Converters

For our data downsizer between a wide slave port of width D_W and a narrow master port of width D_N , the critical path goes through the data selection and steering logic, scaling logarithmically with the downsize ratio $\mathcal{O}(\log(D_W/D_N))$. The area is $\mathcal{O}(D_N D_W)$, the first term accounting for the multiplexing logic for data selection and steering, and the second accounting for the registers that hold a wide beat for data packing on the write data channel. Fig. 19a (left side) shows the area and timing characteristics of our downsizer: for a slave port of width 64 bits and a master port of width 8 to 32 bits, the critical path decreases with increasing width of the master port (and decreasing downsize ratio), from 390 to 365 ps, while the area grows linearly from 23 to 25 kGE.

For the data upsizer between a narrow slave port of width D_N and a wide master port of width D_W , the critical path goes through the data selection logic and the round-robin arbiter, scaling linearly with the number of read upsizers R and logarithmically with the upsize ratio, $\mathcal{O}(R \log(D_W/D_N))$. The area of the upsizer scales with $\mathcal{O}(R D_N D_W)$, compounding the effect of the multiplexing logic for data selection and steering, D_N , and of the $R D_W$ -bit registers holding wide beats for data serialization on the read data channel. Fig. 19a (right side) shows the area and timing characteristics of our upsizer: for a slave port of width 64 bits and a master port of width 128 to 512 bits, the critical path increases with the increasing upsize ratio, from 380 to 405 ps, while the area increases from 27 to 35 kGE. Fig. 19b shows the area and timing characteristics of the data upsizer from 64 to 128 bits, for 1 to 8 read upsizers. These have an important effect on the area and critical path of the upsizer. The critical path of the upsizer increases linearly from 380 to 485 ps, while the area increases from 27 to 59 kGE.

3.5 Clock Domain Crossing

The area of the CDC scales linearly with the address, data, and ID widths. For 64 bit address and data width, 6 bit ID width, and a slave port clock frequency of 1 GHz, the CDC area is 27 kGE for master port clock frequencies from 0.1 to

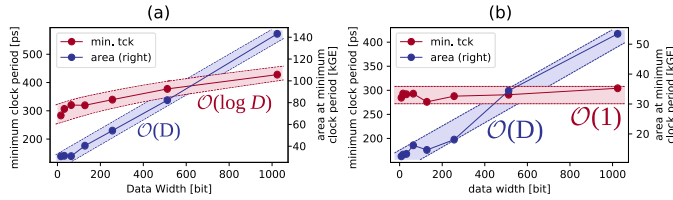


Fig. 20. Minimum clock period and corresponding area in GF22FDX of (a) our DMA engine for 16 to 1024 bit data width, and (b) our simplex on-chip memory controller for 8 to 1,024 bit data width.

2 GHz. Above that, the area increases exponentially but only up to 31 kGE for 5.5 GHz at the master port.

3.6 Data Streaming: DMA Engine

The area of the DMA engine scales with $\mathcal{O}(D)$, where D is the data width, due to the linearly growing alignment buffer. The critical path is dominated by the barrel shifter, which scales with $\mathcal{O}(\log D)$. For 16 to 1,024 bit data width (Fig. 20), the critical path increases logarithmically from 290 to 400 ps and the area increases linearly from 25 to 141 kGE. As the DMA engine uses the same ID for all transactions, the ID width affects neither area nor critical path.

3.7 On-Chip Memory Controllers

3.7.1 Simplex Memory Controller

For a simplex on-chip memory controller with a data width of D , the critical path is constant and found between the command slave channels and the memory master port. The critical path does not depend on D as the transformation of commands does not depend on the data width. Fig. 20b shows the area and timing characteristics: The area scales linearly with $\mathcal{O}(D)$ from 13 to 53 kGE; this linear dependency is caused by the dominant read response buffers needed for response path decoupling. The critical path remains roughly constant around 290 ps. The ID width has no impact on the critical path, as the simplex controller handles all commands in order and only buffers the ID for the response. The area scales with $\mathcal{O}(I)$ due to these buffers.

3.7.2 Duplex Memory Controller

The critical path of the duplex controller goes from the slave port command channels through the demultiplexer, one simplex memory controller, and the logarithmic memory interconnect to a memory command port. For a data width of D and B memory master ports, it scales with $\mathcal{O}(\log D)$. The area is composed of the demultiplexer, the two simplex memory controllers, and the logarithmic interconnect, and thus scales with $\mathcal{O}(B + D)$. Fig. 21 shows the area and

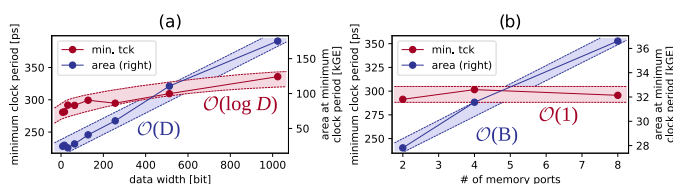


Fig. 21. Minimum clock period and corresponding area of our duplex on-chip memory controller in GF22FDX: (a) for 8 to 1024 bit data width and two memory master ports, and (b) for 64 bit data width and 1 to 8 memory master ports.

TABLE 1
Overview of the Complexity of Our Network Modules

	Critical Path	Area
Multiplexer	$\mathcal{O}(\log S)$	$\mathcal{O}(S)$
Demultiplexer	$\mathcal{O}(M + I)$	$\mathcal{O}(M + 2^I)$
Crossbar	$\mathcal{O}(M + I)$	$\mathcal{O}(MS + 2^I S)$
Crosspoint	$\mathcal{O}(M + I)$	$\mathcal{O}(M + 2^I)$
ID Remapper	$\mathcal{O}(\log I + \log U + \log T)$	$\mathcal{O}(U(I + \log T + \log U))$
ID Serializer	$\mathcal{O}(\log U_M + \log T)$	$\mathcal{O}(U_M + T)$
Data Upsizer	$\mathcal{O}(R \log(D_W/D_N))$	$\mathcal{O}(RD_W D_N)$
Data Downsizer	$\mathcal{O}(\log(D_W/D_N))$	$\mathcal{O}(D_W D_N)$
DMA Engine	$\mathcal{O}(\log D)$	$\mathcal{O}(D)$
Simplex Mem. Ctrl.	$\mathcal{O}(1)$	$\mathcal{O}(D)$
Duplex Mem. Ctrl.	$\mathcal{O}(\log D + \log B + I)$	$\mathcal{O}(D + B + 2^I)$

Legend: M = number of master ports; S = number of slave ports. D = data width; D_W = data width of the wide interface; D_N = data width of the narrow interface; I = ID width; U = concurrent unique IDs; U_M = concurrent unique IDs at the master port; T = concurrent transactions per ID. B = number of memory master ports. R = number of read upsizers.

timing characteristics of our duplex memory controller: For $D = 8$ to 1,024 bit data width and $B = 2$ memory master ports (Fig. 21a), the critical path increases logarithmically from 280 to 330 ps, and the area increases linearly from 20 to 175 kGE. For $D = 64$ bit data width and $B = 2$ to 8 memory master ports (Fig. 21b), the critical path stays constant around 300 ps and the area scales with $\mathcal{O}(B)$ from 28 to 34 kGE. Regarding the ID width I , the complexity is defined by the demultiplexer.

3.8 Complexity Overview and Summary

An overview of the asymptotic complexity of our network modules is shown in Table 1. The critical path of all modules scales at worst linearly in their parameters, for most modules and parameters even logarithmically. As the absolute results of the minimum clock period show, the critical path of all modules remains below 500 ps post-topographical-synthesis in the large design space we evaluated. This shows our modules are suited for a wide range of target frequencies and bandwidths, up to 2 GHz. When even higher frequencies are required, most modules can be parametrized to have a critical path below 330 ps, which would allow to clock them up to 3 GHz. The area of most modules scales linearly in their parameters, with the notable exception of the ID width, which causes an exponential growth of the demultiplexer and all modules containing it. As the absolute results show, most modules fit a few tens of kGE when not pushed to the highest possible clock frequency and parametrization. Even more complex modules, such as a 4×4 crossbar with up to 256 independent concurrent transactions, fit in a modest 100 kGE when clocked at 2.5 GHz.

Power is another important aspect of on-chip networks. This paper focuses on architecture, performance, and area, but our platform supports all state-of-the-art power optimization techniques (e.g., architectural power gating). Even for complex and high-performance instances such as the mentioned 100 kGE crossbar, the power consumption is in the order of just 35 mW under full load at 2.5 GHz, which would typically be less than 10% of the power consumed by processor cores operating at a similar frequency; for instance, an ARM Cortex-A72 in a 16 nm FinFET technology at 2.5 GHz consumes around 1 W [13].

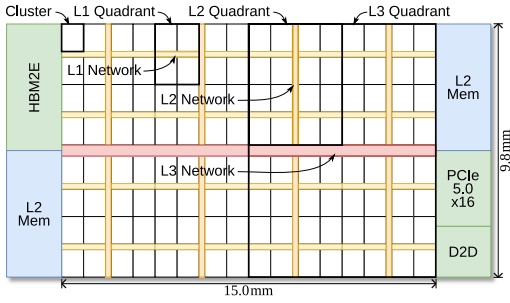


Fig. 22. Conceptual floorplan of one Manticore [14] chiplet die.

While module-wise results are important to show the complexity and trade-offs in the microarchitecture of our on-chip communication platform, they of course cannot show the full picture of a real on-chip network. In the next section, we analyze a full on-chip network.

4 FULL-SYSTEM CASE STUDY

In this section, we design, implement, and evaluate the on-chip network of a many-core floating-point accelerator, using the modules presented in this paper. We use the technology and synthesis flow described in Section 3 and additionally implement the networks with Cadence Innovus 19.1.

The *Manticore* architecture [14] is a state-of-the-art many-core processor for high-performance, high-efficiency, data-parallel floating-point computing. A Manticore accelerator consists of four chiplet dies on an interposer. Each chiplet, shown in Fig. 22, contains 1,024 cores grouped in 128 clusters, one 8 GiB HBM2E controller and PHY, 27 MiB L2 memory, one PCIe 5.0 x16 controller and PHY, and three die-to-die link (D2D) PHYs to the other chiplets. Each cluster contains eight small 32-bit integer RISC-V cores, each controlling a large double-precision floating-point unit (FPU), and 128 KiB L1 memory organized in 32 SRAM banks. As primary means for moving data into and out of L1, each cluster contains two of our DMA engines (Section 2.6, one for reads and one for writes), which are attached to the L1 memory and control a 512-bit-wide master port. DMA engines in other clusters can access the L1 memory through an additional 512-bit-wide slave port. Each cluster has a 64-bit master port to let its cores access external memory and a 64-bit slave port to let cores in other clusters access its L1 memory. Four clusters form an L1 quadrant, four L1 quadrants form an L2 quadrant, four L2 quadrants form an L3 quadrant, and two L3 quadrants form a chiplet. Manticore has been introduced in [14] without disclosing its on-chip network. In the remainder of this section, we describe the design, implementation, and performance of Manticore's on-chip network.

4.1 Network Design

Manticore's on-chip network is designed with four main goals: **(D1)** High bandwidth between units within the same quadrant for effective local data sharing. **(D2)** High bandwidth between the chiplet-level I/Os (i.e., HBM2E, PCIe, D2D) and any cluster for effective data input and output. **(D3)** Low latency between any two cores for efficient concurrency. **(D4)** Minimal interference between the wide bursts of the DMA engine and the word-wise accesses of the cores for maximum network utilization. The network, shown in

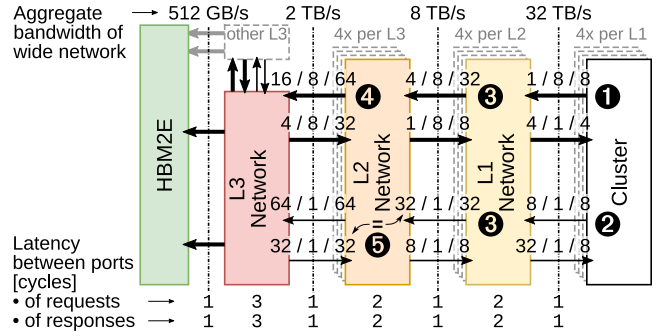


Fig. 23. Manticore's on-chip network. Each arrow represents a bundle, with the arrow head pointing in the direction of the command channels. Fat arrows mean 512 bit data width, thin arrows 64 bit. Numbers above arrows indicate maximum transaction concurrency in the form *unique IDs / transactions per ID / total transactions per bundle* (reads and writes separate).

Fig. 23, has the following properties to meet these goals: (1) Physically separate networks for traffic by DMA engines and cores to meet **(D4)**. (2) Tree topology to meet **(D2-3)**. (3) Fully-connected crossbars within each quadrant to meet **(D1)**. (4) The same data width and frequency for each bundle throughout the DMA network, to meet **(D2)**. The clock frequency of the entire network and all its endpoints (i.e., cores, DMA engines, L2 memory controllers, and HBM2E controller) is 1 GHz. The data width of the DMA network is set to 512 bit, which corresponds to one of the four ports into the HBM2E controller. Therefore, saturating the full HBM2E bandwidth requires concurrent transactions from only four DMA engines in different L2 quadrants. The data width of the core network is set to 64 bit, which is native for the load/store unit of a core.

The concurrency of transactions is another important aspect of the network design. The numbers above an arrow in Fig. 23 define the number of concurrent unique IDs, transactions per ID, and total transactions per bundle (reads and writes separate), respectively. ID width converters are placed in the network where required to reduce the ID width. Starting at the cluster, each DMA engine is in-order (thus has a single ID) and can have up to 8 outstanding transactions ①. Transactions by the 8 cores in the cluster are independent, and each core can have at most 1 outstanding transaction ②. The L1 network maintains the independence of all DMA and core transactions, and the number of unique IDs expands accordingly, as do the total transactions ③. The L2 network maintains the independence of DMA transactions but limits their total below the sum of the incoming ports with ID remappers ④. The reason is that the maximum roundtrip latency at this level is 60 cycles, so a higher number of concurrent transactions would not increase bandwidth or utilization. The concurrency on downlinks is generally constrained with ID remappers to match that of an uplink into the lower network, e.g., ⑤. This means each network level can handle transactions from the uplink slave port in the same way as transactions from downlink slave ports.

4.2 Network Microarchitecture and Implementation

The microarchitecture and physical dimensions of one L1 and L3 network are shown in Fig. 24. (The L2 network is very similar to the L1 and omitted for brevity.) For the L1 network, the downlink ports are in the left third of each

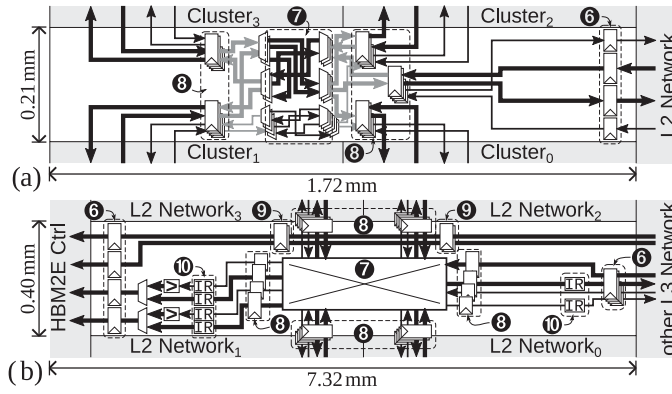


Fig. 24. Microarchitecture and dimensions of Manticore's on-chip network. (a) L1 network. (b) L3 network. Only select connections are drawn for reasons of lucidity.

cluster, close to the cluster's memory and internal interconnect, and the uplink port is in the middle of the narrow side. For the L2 and L3 network, the downlink ports are at one quarter of the wide side (determined by the lower network level), and the uplink port is in the middle of the narrow side. To isolate the timing closure of individual network levels, we cut all paths at the uplink ports ⑥. Correspondingly, all downlink inputs are driven by FFs and all downlink outputs drive FFs. There are two central challenges in the physical implementation of the networks. First, the extremely wide aspect ratio: while one wide dimension is determined by the side length quadrant, the other dimension should be as narrow as possible to minimize the area of the network. Second, routing and wire congestion: each of the five interfaces has ca. 3,300 separate wires, and each network level is fully connected. Routing the wires of a single interface horizontally occupies a height of ca. 100 μm on all three metal layers available for inter-cell horizontal routing. To mitigate congestion, the crossbar, with its fanout of wires between demultiplexers and multiplexers, should be placed and routed as compact as possible ⑦. The crossbar nonetheless incurs a significant combinational delay. To accommodate this despite the long distances due to the extreme aspect ratio, we insert registers around the crossbar ⑧. In contrast to pipelining inside the crossbar, much fewer registers are required, which again benefits the compact layout of the crossbar. In the L3 network (Fig. 24b), pairs of L2 networks share one port on the HBM2E controller. Cores on the narrow network access the wide HBM ports through data width converters. Because the HBM2E controller is located on the left side of the chiplet, the left L3 network simply feeds two connections from the right L3 network through pipeline registers to the controller ⑨. ID remappers are used to reduce ID widths according to the concurrency design ⑩.

The implementation results of Manticore's on-chip network are listed in Table 2. We have been able to close timing and DRC of the entire network after place and route at 1 GHz. For this, we first loosely constrained the narrow dimension to determine the required number of pipeline registers around the crossbar, then we reduced the narrow dimension until the design could no longer be routed without failing timing or DRC. As the high routing densities show, the area of each network level is mainly determined

TABLE 2
Implementation Results of Manticore's on-chip Network

	Unit	L1	L2	L3	Entire Network
Clock Frequency	[GHz]	1.00	1.00	1.00	1.00
Routing Density*	[%]	59.6	49.6	45.7	—
Area per Inst.	[mm ²]	0.41	1.40	2.99	30.43
Power per Inst.	[mW]	8.1	12.8	17.2	396.0
# Insts. per Chiplet		32	8	2	1
Area per Chiplet	[mm ²]	13.21	11.23	5.98	30.43
Area per Chiplet [†]	[%]	9.05	7.69	4.10	20.84
Area per Core+FPU	[μm^2]	12900	10970	5840	29710
Power per Chiplet	[mW]	259.2	102.4	34.4	396.0

*Routing density along wider dimension (i.e., where routing is denser).

[†]Relative to chiplet area without I/O controllers and PHYs.

by the available routing channels. The total area of the network is 30.43 mm², which is 20.84% of the chiplet area without I/O controllers and PHYs. Put differently, Manticore's entire high-bandwidth, low-latency, hierarchical on-chip network requires 29710 μm^2 per core. This is merely about the same area as one core (without any cache) and FPU, which are highly area-efficient. The total power of the network is 396 mW, which means only 0.4 mW per core (or less than 5% extra power per core).

4.3 Network Performance

We characterize the performance of two fundamental neural network (NN) layers based on RTL simulations and analytical calculations, with a focus on the impact of the on-chip network. The two layers, a convolutional layer and a fully-connected layer, together account for 95 to 99% of the floating-point operations (FLOPs) in MLT. The following is a condensed description of the NN implementation on Manticore, a comprehensive description is available as online supplement.²

A convolutional NN layer transforms a 3D input volume (aka "feature map") to a 3D output volume through convolutions with filter kernels. More precisely, each input volume with dimensions $W_I \times W_I \times D_I$ is padded with P zeros in the spatial dimensions (resulting in $(W_I + 2P) \times (W_I + 2P)$) and then convolved with K filter kernels with dimension $F \times F \times D_I$ in a stride S to produce an output volume with dimensions $W_O \times W_O \times D_O$, where $W_O = (W_I + 2P - F)/S + 1$ and $D_O = K$. We set $W_I = 32$, $D_I = 128$, $K = 128$, $F = 3$, $P = 1$, and $S = 1$. Therefore, $W_O = 32$ and $D_O = 128$. In the baseline implementation, each cluster computes one depth slice (aka "channel") of the output volume (of dimensions $W_O \times W_O$) at a time. As the entire input volume does not fit into the local memory of a cluster, the cluster loads a stack of depth slices of the input volume at a time. Thus, each cluster needs to load the entire input volume once per output depth slice. As the first result column in Table 3 shows, this implies a very low operational intensity and entails that performance is bound by the HBM memory bandwidth. One strategy to alleviate this is to let each cluster compute a stack of depth slices of the output volume. As the input depth slices can be reused for multiple output depth slices, this reduces the amount of data transferred per computation. For a stack of 8 output depth slices (second column), the

2. <https://arxiv.org/pdf/2104.08009.pdf>

TABLE 3
Performance of Manticore for Different NN Layer Implementations

Figure	Unit	Convolution			Fully Connected
		base	stacked	pipe'd	
Op. Intensity	[dpflop/B]	2.2	15.9	15.9	7.9
HBM BW	[GB/s]	262*	98	6	222
L3 Agg. BW	[GB/s]	262	98	6	222
L2 Agg. BW	[GB/s]	262	98	25	222
L1 Agg. BW	[GB/s]	262	98	98	222
Performance	[Gdpflop/s]	571	1638 [†]	1638 [†]	1638 [†]

*Of which 256 GB/s are on the read channel, which is its maximum.

[†]This corresponds to an FPU utilization of ca. 80%, which is the maximum all 8 FPUs in a cluster can sustain for real kernels.

operational intensity is sufficiently high that the performance becomes compute-bound. To save even more off-chip bandwidth without sacrificing performance, the hierarchical network can be used to form a processing pipeline where clusters obtain their input depth slice from another cluster instead of off-chip memory. The third column shows that when all 16 clusters within one L2 quadrant form such a pipeline, the off-chip memory traffic can be massively reduced while performance is maintained. Traffic is also reduced on the L2 and L3 networks because data, once it is in the local memory of a cluster, is mainly transferred through the L1 networks.

A fully-connected NN layer transforms an input volume with dimensions $W_I \times W_I \times D_I$ to an output volume with dimensions $1 \times 1 \times D_O$. As any fully-connected layer can be represented as convolutional layer, we stick to the introduced notation and operations and set $W_I = 32$, $D_I = 128$, $K = 128$, $F = 32$, $P = 0$, and $S = 1$. Therefore, $W_O = 1$ and $D_O = 128$. As each filter parameter is used only once per input-output volume pair, fully-connected layers are usually implemented by transforming a *batch* of B input volumes to a batch of B output volumes; we use $B = 32$. Our implementation parallelizes the input depth slices over the clusters. Before the parallel section, each cluster allocates a private output volume and initializes it to zero. In the parallel section, the cluster first loads the entire batch of one depth slice of the input volume and then loops over the output depth slices. Within that loop, the cluster loads the filter parameters for the current pair of input and output depth slices and then enters an inner loop over the batch. Within the inner loop, the cluster multiplies the input depth slice of a batch element with the loaded filter parameters element-wise and then accumulates all products to a single value, which it adds to the output element for the current output depth slice and batch element. After the parallel section, the private output volumes of all clusters are reduced by summation to a single output volume, which contains the contributions of all input depth slices. As the last column in Table 3 shows, this implementation is compute-bound. There is no communication between the clusters in the parallel region because there is no data common to any two clusters. As such, the hierarchy of the network is not exploited, but the high bandwidth between HBM and any cluster is: it allows reaching compute-boundedness already for a batch size of 32. Larger batch sizes further reduce the off-chip bandwidth.

5 RELATED WORK

Network-on-chip (NoC) topologies, routing algorithms, flow control schemes, and router architectures have been subject to a vast amount of research (see [1], [2], [3], [4], [15], [16] for detailed reviews). Important conclusions from this research are that the optimal on-chip network topology highly depends on the target application and computer architecture, and that routing strategies and flow control schemes are intertwined with the communication protocol, which all connected modules need to adhere to. Thus, we do not try to innovate in this field. Rather, the modules in our platform allow to build an on-chip network with arbitrary topology that adheres to a state-of-the-art, industry-standard protocol, following the paradigm put forward by application-specific NoC research efforts (see [17] for an up-to-date survey). Additionally, our elementary modules allow to design custom network modules, including custom endpoints such as caches and memory controllers, without having to deal with all protocol intricacies. To the best of our knowledge, our work is the first on-chip communication platform that offers elementary modules smaller than crossbars or switches.

Non-coherent on-chip communication is central for heterogeneous, accelerator-rich SoCs [18]. Protocols similar to AMBA AXI5 [8], which our platform directly supports, are IBM's CoreConnect [19], Silicore's Wishbone [20], Accellera's Open Core Protocol (OCP) [21], and SiFive's TileLink Uncached Heavyweight (TL-UH) [22]. They all, like AXI, are royalty-free standards. CoreConnect, Wishbone, and OCP provide a subset of the features of AXI5, and while they had been used in the past, they are nowadays not nearly as widely used as AXI. TL-UH, like AXI5, supports burst transactions, multiple outstanding transactions, and transaction reordering and uses valid-ready flow control. TL-UH has stricter forward progress requirements than AXI5, which our modules could also fulfill. While the specifications define protocols for on-chip communication, they do not describe the architecture of network modules implementing them; that is an important contribution of our work. The OpenSoC Fabric [23] is an open-source implementation of a custom non-coherent protocol, with an interface to AXI-Lite in development. AXI-Lite does not support bursts or transaction reordering and is therefore not suited for high-performance communication. The ESP project [24] provides an open-source implementation of a 2D-mesh NoC with a custom protocol. Similarly, the non-coherent BaseJump Manycore Accelerator Network, which has first been used in the Celerity chip [25], adheres to a custom protocol and is designed for 2D-mesh networks. In contrast, our platform is topology-agnostic and adheres to an industry-standard protocol. An overview of on- and off-chip interconnects for NN accelerators is presented in [26]. They highlight the need for non-mesh topologies in NN accelerators, to which we contribute with our case study and topology-independent platform.

Commercial intellectual property (IP) offerings for AXI exist from multiple vendors, and we compare with them in Table 4. Our work is the only one (1) whose architecture is fully disclosed in literature, (2) whose register-transfer level (RTL) code is open-source and modifiable, enabling, e.g.,

TABLE 4
Commercial IP Offerings for AXI Compared With This Work

Product	Disclosable Open Public Prod. Spec.	RTL Architecture	RTL Open-Source AT Results	Elms. for Custom ASIC Usable on FPGA	Data Width [bit]	Max. Concur. Txns.	ID Width Conv.	OCM Controller DMA Engine
Arm [27] NIC-400	✓	✗	✗	✗	32–256	1–127	✓	✗
Arteris [28] FlexNoc	✗	✗	✗	✗	?	?	?	?
Synopsys [29] DW_axi	✗	✗	✗	✗	≤ 512	?	?	✗
Xilinx [30] AXI LogiCore	✓	✗	✓	✗	32–1024	1–32	✓	✗
This Work	✓	✓	✓	✓	8–1024*	1–128 [†]	✓	✓

⊛ = not disclosed publicly. ⌘ = licensed separately. † Auto-inferred inside crossbar, cannot be customized or instantiated standalone. * Limited by AXI standard, larger data widths theoretically possible. ‡ Range evaluated in this work, larger values theoretically possible.

the exploration of arbitration algorithms with certain guarantees inside standard-compliant networks, and (3) whose area and timing (AT) characteristics may be disclosed not only on FPGAs. Despite its research origin, the implementation behind this work powers on-chip communication of an increasing number of ASICs (e.g., [14], [31]). From a technical perspective, our work is the only one that offers elementary modules (i.e., *network* (de)multiplexers) that can be used to build custom AXI-compliant IP modules without having to deal with all intricacies of the protocol, instead of only crossbars as finest-granularity modules. Additionally, our work supports all standard-defined data widths, supports the highest number of concurrent transactions, and comes with communication modules such as ID width converters, a DMA engine, and on-chip memory controllers,³ which are licensed separately or not available at all from commercial vendors.

Cache-coherent on-chip communication protocols currently in use include Intel’s UltraPath Interconnect [32], AMD’s scalable data fabric [33], IBM’s Power9 on-chip interconnect [34], AMBA AXI Coherency Extensions (ACE) [8], AMBA5 Coherent Hub Interface (CHI) [35], and TileLink Cached (TL-C) [22]. ACE and TL-C are extensions of AXI and TL-UH, respectively. As such, our platform could be extended for coherent communication by adding channels, transactions, and properties defined by these specifications. Under the ordering rules (O1–3), coherence transactions cannot use the existing channels of regular transactions: In coherent communication, a regular transaction can entail coherence transactions, which must complete before the regular transaction can complete. Guaranteeing this would require different ordering rules. The other protocols are standalone specifications with very different properties. For instance, we refer to [36] for an open-source bridge for connecting to CHI from AXI. With such a bridge, our platform can connect to a coherent system interconnect if needed, possibly extending to multiple chips. Coherency in on-chip

networks has been studied extensively in research, e.g., [37], [38], [39]. A prominent system example is SCORPIO [40], where a coherent mesh NoC interconnects 36 homogeneous cores on a die. Their work focuses on the NoC and router architecture for a coherent homogeneous multi-core, while we design an end-to-end non-coherent on-chip communication platform suitable for heterogeneous many-cores. Generators for cache-coherent on-chip networks have been presented in multiple works: Open2C [41] contains a library of modules for coherent networks written in Chisel. The Rocket chip generator [42] constructs SoCs written in Chisel, and the coherent NoC adheres to TL-C. OpenPiton [43] generates tile-based manycore processors with a 2D mesh coherent NoC. One tile has an area of 1.17 mm² when targeting IBM’s 32 nm SOI process at 1 GHz. Of the tile area, 2.7% are occupied by the 5 × 5 NoC router. Accounting for one full technology node difference, the equivalent area in GF22FDX would be ca. 80 kGE, which is about the same size as a 5 × 5 configuration of our crosspoint %. Open2C and OpenPiton implement a custom protocol, which complicates connectivity with third-party modules, whereas we adhere to an industry-dominant protocol. The modules in our work are implemented in synthesizable SystemVerilog, so they could be integrated into a higher-level generator as well.

6 CONCLUSION

This first fully open-source⁴ platform for high-performance on-chip communication enables the construction of heterogeneous many-core and accelerator-rich SoCs independent of proprietary on-chip networks IPs. The platform advances the technical state of the art through two main contributions: First, network (de)multiplexers as elementary components make the design and verification of custom network modules substantially easier. Second, an end-to-end palette of modules from a DMA engine to on-chip memory controllers, including data and ID width converters, as well as the widest range of data widths and concurrent transactions enables new designs. For example, we designed and implemented a state-of-the-art 1024-core MLT accelerator in a modern 22 nm technology, where our communication fabric provides 32 TB/s cross-sectional bandwidth at only 24 ns round-trip latency between any two cores. Future work enabled by our platform includes design space exploration and optimization of on-chip networks, networks designed for stringent application constraints (e.g., arbitration guarantees for real-time execution), and co-integrated cache-coherent and non-coherent networks.

REFERENCES

- [1] N. Jerger *et al.*, *On-Chip Networks: Second Edition*. San Rafael, CA, USA: Morgan & Claypool, 2017.
- [2] W. Dally *et al.*, *Principles and Practices of Interconnection Networks*. San Mateo, CA, USA: Morgan Kaufmann, 2003.
- [3] L. Benini *et al.*, *Networks on Chips: Technology and Tools*. San Mateo, CA, USA: Morgan Kaufmann, 2006.
- [4] S. Kundu *et al.*, *Network-on-Chip: The Next Generation of System-on-Chip Integration*. Boca Raton, FL, USA: CRC Press, 2018.

3. Our platform additionally includes a last-level cache (LLC), which is not described in this paper due to space constraints but is available in our open-source repository.

4. SystemVerilog source code available under a permissive open-source license at <https://github.com/pulp-platform/axi>.

- [5] Qualcomm Inc., "Snapdragon 865 5G mobile platform," 2020. [Online]. Available: https://www.qualcomm.com/system/files/document/files/prod_brief_qcom_sd865_5g.pdf
- [6] B. Wheeler, "Tomahawk 4 switch first to 25.6 Tbps," *Microprocessor Report*, 2019. [Online]. Available: https://www.linleygroup.com/newsletters/newsletter_detail.php?num=6104
- [7] R. Smith, "NVIDIA Ampere unleashed: NVIDIA announces new GPU architecture, A100 GPU, and accelerator," *AnandTech*, 2020. [Online]. Available: <https://www.anandtech.com/show/15801/nvidia-announces-ampere-architecture-and-a100-products>
- [8] AMBA AXI and ACE Protocol Specification Issue F.b, Arm Ltd., 2017. [Online]. Available: <https://developer.arm.com/documentation/ih0022/fb/>
- [9] E. G. Coffman *et al.*, "System deadlocks," *ACM Comput. Surv.*, vol. 3, pp. 67–78, 1971.
- [10] R. W. Apperson, Z. Yu, M. J. Meeuwsen, T. Mohsenin, and B. M. Baas, "A scalable dual-clock FIFO for data transfers between arbitrary and halttable clock domains," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 15, no. 10, pp. 1125–1134, Oct. 2007.
- [11] C. E. Cummings, "Clock domain crossing (CDC) design & verification techniques using SystemVerilog," in *Proc. Conf. Synopsys Users Group*, 2008, pp. 29–35.
- [12] A. Strano, D. Ludovici, and D. Bertozzi, "A library of dual-clock FIFOs for cost-effective and flexible MPSoC design," in *Proc. Int. Conf. Embedded Comput. Syst.: Architectures Model. Simul.*, 2010, pp. 20–27.
- [13] A. Frumusanu, "Cortex A72 performance and power," *AnandTech*, 2016. [Online]. Available: <https://www.anandtech.com/show/9878/the-huawei-mate-8-review/3>
- [14] F. Zaruba, F. Schuiki, and L. Benini, "Manticore: A 4096-core RISC-V chiplet architecture for ultra-efficient floating-point computing," in *Proc. IEEE Hot Chips 32 Symp.*, 2020, pp. 1–24.
- [15] S. Pasricha *et al.*, *On-Chip Communication Architectures: System on Chip Interconnect*. Amsterdam, Netherlands: Elsevier Science, 2010.
- [16] J. Flich *et al.*, *Designing Network On-Chip Architectures in the Nano-scale Era*. Boca Raton, FL, USA: CRC Press, 2010.
- [17] A. Cilaro *et al.*, "Design automation for application-specific on-chip interconnects: A survey," *Integration*, vol. 52, pp. 102–121, 2016.
- [18] D. Giri, P. Mantovani, and L. P. Carloni, "Accelerators and coherence: An SoC perspective," *IEEE Micro*, vol. 38, no. 6, pp. 36–45, Nov./Dec. 2018.
- [19] *CoreConnect processor local bus specification*, IBM Inc., 2007. [Online]. Available: <https://web.archive.org/web/20090211133230/http://www-01.ibm.com/chips/techlib/techlib.nsf/techdocs/3BBB27E5BCC165BA87256A2B0064FFB4>
- [20] *Wishbone B4 SoC interconnection architecture*, Silicore Corp., 2010. [Online]. Available: https://cdn.opencores.org/downloads/wbspec_b4.pdf
- [21] Accellera Inc., *Open Core Protocol Specification Release 3.0*, 2013. [Online]. Available: <https://www.accellera.org/downloads/standards/ocp>
- [22] *SiFive TileLink Specification v1.8.0*, SiFive Inc., 2019. [Online]. Available: <https://github.com/chipsalliance/omniextend/blob/master/OmniXtend-1.0.3/spec/TileLink-1.8.0.pdf>
- [23] F. Fatollahi-Fard, D. Donofrio, G. Michelogiannakis, and J. Shalf, "OpenSoC Fabric: On-chip network generator," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw.*, 2016, pp. 194–203.
- [24] D. Giri, P. Mantovani, and L. P. Carloni, "NoC-based support of heterogeneous cache-coherence models for accelerators," in *Proc. 12th IEEE/ACM Int. Symp. Netw.-on-Chip*, 2018, pp. 1–8.
- [25] S. Davidson *et al.*, "The Celerity open-source 511-core RISC-V tiered accelerator fabric: Fast architectures and design methodologies for fast chips," *IEEE Micro*, vol. 38, no. 2, pp. 30–41, Mar./Apr. 2018.
- [26] S. M. Nabavinejad, M. Baharloo, K.-C. Chen, M. Palesi, T. Kogel, and M. Ebrahimi, "An overview of efficient interconnection networks for deep neural network accelerators," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 10, no. 3, pp. 268–282, Sep. 2020.
- [27] *ARM CoreLink NIC-400 TRM, Revision G*, Arm Ltd., 2016. [Online]. Available: <https://developer.arm.com/documentation/ddi0475/g>
- [28] J.-J. Leclerc *et al.*, "Application driven network-on-chip architecture exploration and refinement for a complex SoC," *Des. Autom. Embedded Syst.*, vol. 15, pp. 133–158, Jun. 2011.
- [29] Synopsys Inc., "DesignWare IP solutions for AMBA AXI 4," 2018. [Online]. Available: https://www.synopsys.com/dw/doc.php/ds/i/dw_amba_solutions.pdf
- [30] *AXI Interconnect v2.1 LogiCORE IP Product Guide*, Xilinx Inc., 2017. [Online]. Available: https://www.xilinx.com/support/documentation/ip_documentation/axi_interconnect/v2_1/pg059-axi-interconnect.pdf
- [31] F. Zaruba, F. Schuiki, S. Mach, and L. Benini, "The floating point trinity: A multi-modal approach to extreme energy-efficiency and performance," in *Proc. 26th IEEE Int. Conf. Electron. Circuits Syst.*, 2019, pp. 767–770.
- [32] D. Mulnix, "Intel Xeon processor scalable family technical overview," Intel Corp., 2017. [Online]. Available: <https://software.intel.com/content/www/us/en/develop/articles/intel-xeon-processor-scalable-family-technical-overview.html>
- [33] T. Burd *et al.*, "Zeppelin: An SoC for multichip architectures," *IEEE J. Solid-State Circuits*, vol. 54, no. 1, pp. 133–143, Jan. 2019.
- [34] S. K. Sadasivam, B. W. Thompto, R. Kalla, and W. J. Starke, "IBM Power9 processor architecture," *IEEE Micro*, vol. 37, no. 2, pp. 40–51, Mar./Apr. 2017.
- [35] *AMBA5 CHI specification Issue D*, Arm Ltd., 2019. [Online]. Available: <https://developer.arm.com/documentation/ih0050/c/>
- [36] M. Cavalcante *et al.*, "Design of an open-source bridge between non-coherent burst-based and coherent cache-line-based memory systems," in *Proc. 17th ACM Int. Conf. Comput. Frontiers*, 2020, pp. 81–88.
- [37] N. Easley, L. Peh, and L. Shang, "In-network cache coherence," in *Proc. 39th Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2006, pp. 321–332.
- [38] N. D. Enright Jerger, L. Peh, and M. H. Lipasti, "Virtual tree coherence: Leveraging regions and in-network multicast trees for scalable cache coherence," in *Proc. 41st IEEE/ACM Int. Symp. Microarchitecture*, 2008, pp. 35–46.
- [39] N. Agarwal, L. Peh, and N. K. Jha, "In-network coherence filtering: Snoopy coherence without broadcasts," in *Proc. 42nd Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2009, pp. 232–243.
- [40] B. K. Daya *et al.*, "SCORPIO: A 36-core research chip demonstrating snoopy coherence on a scalable mesh NoC with in-network ordering," in *Proc. ACM/IEEE 41st Int. Symp. Comput. Architecture*, 2014, pp. 25–36.
- [41] A. Butko *et al.*, "Open2C: Open-source generator for exploration of coherent cache memory subsystems," in *Proc. Int. Symp. Memory Syst.*, 2018, pp. 311–317.
- [42] K. Asanović *et al.*, "The Rocket chip generator," Univ. California, Berkeley, Berkeley, CA, Tech. Rep. UCB/EECS-2016-17, Apr. 2016.
- [43] J. Balkind *et al.*, "OpenPiton: An open source manycore research framework," in *Proc. 21st Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2016, pp. 217–232.



Andreas Kurth (Student Member, IEEE) received the BSc and MSc degrees in electrical engineering and information technology from ETH Zurich, Switzerland, in 2014 and 2017, respectively. He is currently working toward the PhD degree in the Digital Circuits and Systems Group of Prof. Benini. His research interests include the architecture and programming of heterogeneous SoCs and accelerator-rich computing systems.



Wolfgang Rönninger received the BSc and MSc degrees in electrical engineering and information technology from ETH Zurich, Switzerland, in 2017 and 2019, respectively. He currently works as a research assistant in the Digital Circuits and Systems group of Prof. Benini. His research interests include high-performance on-chip communication networks and general-purpose memory hierarchies.



Thomas Benz received the BSc and MSc degrees in electrical engineering and information technology from ETH Zurich, Switzerland, in 2018 and 2020, respectively. He is currently working toward the PhD degree in the Digital Circuits and Systems Group of Prof. Benini. His research interests include energy-efficient high-performance computer architectures and the design of ASICs.



Florian Zaruba (Student Member, IEEE) received the BSc degree from TU Wien, Austria, in 2014, and the MSc degree from the ETH Zurich, Switzerland, in 2017. He is currently working toward the PhD degree in the Digital Circuits and Systems Group of Prof. Benini. His research interests include design of VLSI circuits and high-performance computer architectures.



Matheus Cavalcante (Student Member, IEEE) received the MSc degree in integrated electronic systems from the Grenoble Institute of Technology (Phelma), France, in 2018. He is currently working toward the PhD degree in the Digital Circuits and Systems Group of Prof. Benini. His research interests include vector processing and high-performance computer architectures.



Luca Benini (Fellow, IEEE) holds the chair of Digital Circuits and Systems at ETH Zurich, Switzerland, and is full professor at the Università di Bologna, Italy. His research interests are in energy-efficient computing systems design, from embedded to high-performance. He has published more than 1000 peer-reviewed papers and five books. He is a fellow of the ACM and a member of Academia Europaea.



Fabian Schuiki received the BSc and MSc degrees in electrical engineering and information technology from ETH Zurich, Switzerland, in 2014 and 2017, respectively. He is currently working toward the PhD degree in the Digital Circuits and Systems Group of Prof. Benini. His research interests include computer architecture, transprecision computing, as well as near-memory and in-memory processing.

▷ For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdsl.